SESSION 6
INTRODUCTION TO EXPRESS.JS FRAMEWORK

Learning Objectives

In this session, students will learn to:

- Describe the Express.js framework
- Explain the architecture of Express.js
- List the advantages of using Express.js
- Explain the working of Express.js

Frameworks are tools that help in building applications. Frameworks are similar to blueprints of houses. These blueprints provide the plan for building the house including the layout and various measurements of the rooms. Following the plans ensure fast construction and reduces the chances of errors, thereby enhancing the quality of construction. The blueprints also use the language, structure, and symbols that are understood by all builders globally, thereby facilitating easy collaboration among builders, as required. Similarly, frameworks provide libraries and specify rules and guidelines that the developers can use to develop applications faster with minimal errors. This improves the quality of the code in their applications. Frameworks also provide a common language and structure to organize the code, making it easier for developers to collaborate with each other.

Express.js is one such framework that can be used for Node.js programming.

This session explains the Express.js framework, its architecture, working, and advantages.

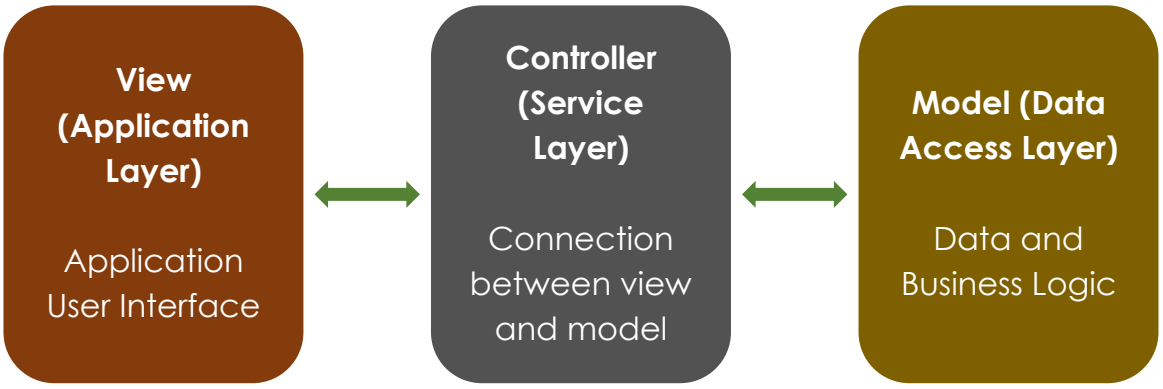
6.1 What is Express.js?

Express.js is a framework that is used for server-side programming with Node.js runtime environment. This framework provides features and libraries that can be used to develop Web applications, mobile applications, and APIs. Express.js can handle different types of Hypertext Transfer Protocol (HTTP) requests, generate dynamic Hypertext Markup Language (HTML), provide middleware functions, and serve static files. Express.js is an unopinionated framework that does not restrict developers to follow a prescribed way of developing applications. It allows developers to be flexible and innovative with their application development using the tools and techniques of their choice. Developers can also extend Express.js to add customized functionalities as required.

Middleware provides services such as authentication, security, and transaction processing to the applications.

6.2 Express.js Architecture

The Express.js framework is made up of three interconnected components: view, controller, and model, spread across three layers: application, service, and data access, respectively.



The **view** component makes up the application layer. It is responsible for generating the HTML content that is sent to the client.

The **model** component makes up the data access layer. It handles the data and the business logic and interacts with the underlying data sources.

The **controller** component makes up the service layer and provides the logic that acts as a bridge between the view and model components.

In Express.js the HTTP requests and responses are handled by the middleware. The middleware is a software that has access to the HTTP requests and responses and includes the next function that is used to move from one request to another. The middleware performs various functions including authentication, validation, and error handling. These functions can be chained together to form a pipeline that handles the requests and responses in a specific order.

6.3 Working of Express.js

Figure 6.1 shows the working sequence of the components of Express.js.

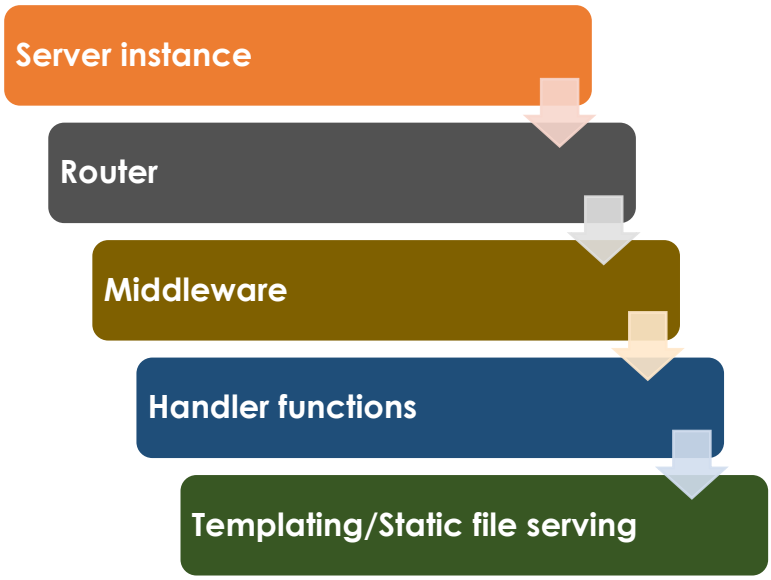


Figure 6.1: Components of Express.js

As shown in Figure 6.1, Express.js creates a server instance first. This instance listens for requests from the client on the specified port. When the instance receives a request, it passes it on to the router. The router, in turn, examines the request to identify the Universal Resource Locator (URL) and the HTTP method, such as `POST`, `GET`, `DELETE`, or `PUT`. Based on the URL and HTTP method, the router determines the handler function to which the request must be passed. The router adds the required details of the handler function and passes on the request to the middleware.

The middleware performs various tasks on the request including authentication, data compression, logging, and then passes on the request to the handler function. The handler function then, processes the request and generates the response. If required, the function involves a templating engine to generate dynamic HTML. The handler function can also access static files, such as images, Cascading Style Sheets (CSS), or JavaScript files, as required for generating the response. After the response is generated, it is sent to the server instance, which, in turn, sends the response to the client.

6.4 Why Use Express.js?

Express.js is a very popular choice with developers for developing Web applications and APIs. This is because it provides various advantages that makes it easier for the developer to build simple and standalone to complex and distributed Web applications. The advantages of using Express.js are:

- **It is a minimalist framework**

  The Express.js framework includes a minimal set of core features. This makes the applications lightweight and efficient. It also allows developers to build applications quickly.

- **It is extensible**

  The Express.js framework allows developers to add custom functionalities with ease. It also provides support for various third-party plugins and modules that can be used to extend the functionality of the framework.

- **It provides enhanced performance**

  The Express.js framework is built on top of Node.js. Therefore, it inherits the event-driven, non-blocking model of Node.js, thereby providing enhanced performance and scalability to handle large number of connections concurrently.

- **It provides router and middleware support**

  The Express.js framework uses routers that can handle different types of HTTP requests. It also uses middleware to intercept and modify the requests and responses by performing tasks such as logging, authentication, error handling, and data compression.

- **It is compatible with several databases**

  The Express.js framework allows developers to work with various popular databases, including MongoDB, MySQL, and PostgreSQL, based on the application requirements. Developers can perform Create, Read, Update, and Delete (CRUD) operations on the databases and manage database transactions.

- **It is compatible with templating engines**

  The Express.js framework can work with various templating engines, such as EJS and Pug, to render dynamic HTML content.

- **It can serve static files**

  The Express.js framework can serve static files including CSS, images, JavaScript files directly from the server.

- **It is easy to use**

  The Express.js framework includes easy and intuitive APIs that developers can learn and use easily. This framework is a popular choice among beginners and experienced developers.

6.5 Installing Express.js

Before installing Express.js, developers must ensure that Node.js has been installed.

To install Express.js, first create a folder to store all the Express.js applications. Express.js must be installed in this folder. Next, open a Command Prompt window and navigate to the newly created folder.

It is recommended that developers must install the `init` package before installing Express.js, although it is not mandatory. Installing the `init` package is especially useful when collaborating with other developers. This is because, the `init` package:

- Creates the `package.json` file, which handles the project dependencies and metadata.
- Stores the project information in the `package.json` file, making it easier to identify and share the project with others.
- Facilitates better project management and organization by providing a consistent and standardized project setup, making collaboration easier.

To install the `init` package, at the command prompt, execute the command as:

```
npm install init
```

Figure 6.2 shows the output of this command.

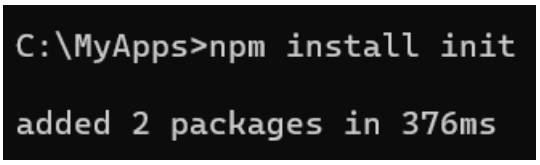


Figure 6.2: Installing the `init` Package

To install Express.js, at the command prompt, execute the command as:

```
npm install express
```

Figure 6.3 shows the output of this command.

```
C:\MyApps>npm install express

added 62 packages, and audited 65 packages in 4s

11 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Figure 6.3: Installing the `express` Package

To verify the installation of Express.js, at the command prompt, execute the command as:

```
npm ls express
```

The output of this command shows the version of Express.js installed, as shown in Figure 6.4.

```
C:\MyApps>npm ls express
MyApps@ C:\MyApps
`-- express@4.18.2
```

Figure 6.4: Version of Express.js

6.6 `request` and `response` Objects

The `request` and `response` objects in Express.js play an important role in handling and serving HTTP requests. The `request` object handles the HTTP requests coming from the clients. It provides information about the request, such as the URL, HTTP method, HTTP header, request body, and IP of the client. All this information helps the developers to handle the requests effectively by taking informed decisions. On the other hand, the `response` object is responsible for generating the responses for the requests and sending them back to the client.

Table 6.1 lists some key properties of the `request` and `response` objects.

Property	Description
`request` object	
`body`	Holds the data sent by the client. It provides the payload of the request in JSON or form-encoded format.
`headers`	Holds all the HTTP headers included in the HTTP requests sent by the client. It provides information, such as content type

Property	Description
	and authorization, that helps determine the preferences and capabilities of the client.
`ip`	Holds the IP address of the client from where the request has been sent. It provides information about the client network and location.
`method`	Provides information about which HTTP method has been used to send the client request. The method could be `GET`, `POST`, `PUT`, or `DELETE`.
`params`	Holds the route parameters that have been extracted from the URL. These parameters help determine the route for processing the client request.
`query`	Contains the query strings appended to the URL included in the client request. These query strings are stored in key-value pairs.
`url`	Stores the complete URL included in the request sent by the client. It contains the structure of the URL including the path, query string, and fragment.
`response` object	
`app`	Holds the reference to the instance of the Express.js application. It can be used to access application-level settings and functionalities related to the response.
`charset`	Specifies the encoding that is used for the response. This encoding ensures that the client can correctly interpret the response.
`headers`	Holds all the HTTP headers to be included in the HTTP response that will be sent to the client. It allows access to settings such as Content-Type, Content-Length, and Cache-Control.
`statusCode`	Specifies the status of the HTTP response. It takes the values such as `200` for OK or `404` for Not Found depending on the result of the request processing.
`type`	Specifies the content type of the response being sent to the client. It takes the values such as `text/html`, `image.png`, or `application/json`.

Table 6.1: Properties of `request` and `response` Objects

Table 6.2 lists the key methods of the `request` and `response` objects.

Method	Description
`request` object	
`accepts(types)`	Used to determine if the client is capable of handling the content format that it has requested
`is (types)`	Used to determine if the client is capable of processing the content format that it has requested
`get(key)`	Used to access specific header or cookie values
`param(name)`	Used to access specific route parameter using its name
`signedCookie(name, secret)`	Used to determine the value of the signed cookie using the secret key
`unmount`	Used to disconnect the request from the router
`query(name)`	Used to access the query string using its name
`response` object	
`download(path, filename)`	Used to provide files, such as images and documents, for download to the client
`json(data)`	Used to convert response to the JSON format and send it to the client
`redirect(url)`	Used to redirect the client to a different URL
`render(view, locals)`	Used to generate dynamic HTML content using the specified template engine and `locals` object
`send(data)`	Used to send specified data to the client as response
`set(name, value)`	Used to set the header of the response using the specified name and value
`status(code)`	Used to set the status of the HTTP response

Table 6.2: Methods of `request` and `response` Objects

6.7 GET and POST Methods

In any Web application, two common methods used to send requests are GET and POST. Table 6.3 provides the differences between these two methods.

GET Method	POST Method
This method is used to send small amount of data to the server.	This method is used to send large amount of data to the server.
This method appends the data to the URL when sending the request.	This method encloses the data in the request body when sending the request.
The requests sent using this method are stored in the browser history and browser log. They can be bookmarked.	The requests sent using this method are not stored in the browser history or browser log. They cannot be bookmarked.
The requests made using this method can only include ASCII characters.	The requests made using this method can include all types of data, including files and images.
The requests made using this method can be cached.	The requests made using this method cannot be cached.
This method offers low security to the request data. So, the request data can be intercepted and hacked easily.	This method offers high security to the request data. So, the request data cannot be easily intercepted or hacked.
This method is generally used to get data from a particular resource, such as getting product lists and user profiles.	This method is generally used to submit data to a particular resource, such as sending a form, uploading a file, editing exiting data, or creating accounts.

Table 6.3: GET and POST Methods

Express.js provides a very simple way to handle these requests. It provides the `app.get` function to define handlers for GET requests and the `app.post` function to define handlers for POST requests.

To handle POST requests, developers must install an additional package, body-parser. To install this package, perform these steps:

1. Open a Command Prompt window.
2. At the Command Prompt, navigate to the folder where Express.js is installed.

3. At the Command Prompt, run the installation command as:

```
npm i express body-parser
```

The command executes and installs the package as shown in Figure 6.5.

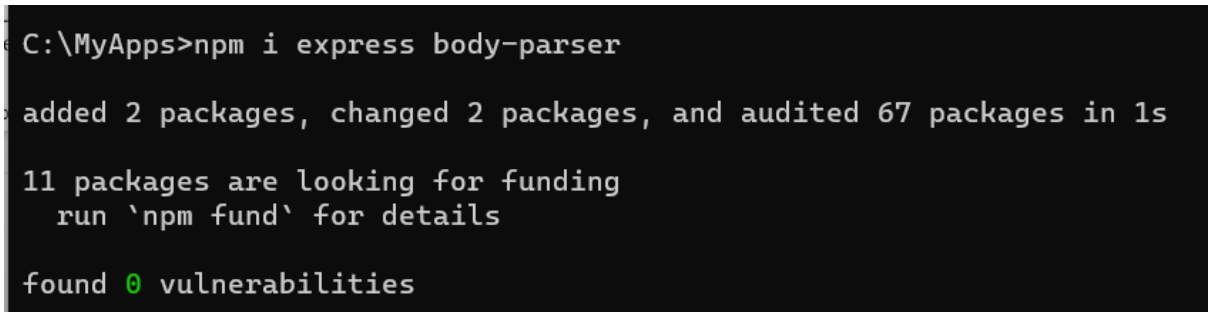


Figure 6.5: Installing the `body-parser` Package

Now, the system is ready for handling `GET` and `POST` requests. For example, consider that there is a Web page `StudentResult` on the local host. This page provides two text boxes to enter the name of the student and marks of the student, respectively. There is also a **Get Result** button on the page. When this button is clicked, the response is received based on the marks of the student. To achieve this using Express.js, perform these steps:

1. To create a Web page, open the VS Code or Notepad application, and type the code given in Code Snippet 1.

Code Snippet 1:

```
<!DOCTYPE html>
<html>
     <title> Student Result </title>
     <body>
          <form action="/result" method="POST">
               Student Name: <input type="text"
name="stuName">
               <br>
               <br>
               Student Marks: <input type="text"
name="stuMarks">
               <br>
               <br>
               <input type="submit" value="Get Result">
          </form>
     </body>
</html>
```

2. In the folder where Express.js has been installed, save this Web page as `StudentResult.html`.

The next step is to create the Express.js application that will use the `app.get` method to open the `StudentResult.html` page when the users navigate to the localhost server. The user can then enter the student's name and marks and click the **Get Result** button. The application will then use the `app.post` method to display the result based on the marks. If the marks is more than 60, it will display `Result: Passed`, else it will display `Result: Failed`.

To create the Express.js application, perform these steps:

1. Open a new file.
2. To create an Express Web server, create a body-parser instance and configure the server to listen on port 3000. To do this, in the file, add the code given in Code Snippet 2.

Code Snippet 2:

```
//Creating an Express Web server
const var_express = require('express');
const var_app = var_express();

//Creating a body-parser instance
const var_parser = require('body-parser');
var_app.use(var_parser.json())
var_app.use(var_parser.urlencoded({ extended: false }))

//Configuring the Web server to listen on port 3000
const port = 3000;
var_app.listen(port, () => {
 console.log(`My Express App is running at
http://localhost:${port}`);
});
```

3. To send the `StudentResult.html` page using the `app.get` method, in the same file, after `const port = 3000`, add the code as shown in Code Snippet 3.

Code Snippet 3:

```
//Serving the StudentResult.html page using app.get()
var_app.get('/', (req, res) => {res.sendFile(__dirname +
'/StudentResult.html')});
```

4. To send the student's result as a response using the `app.post` method, in the same file, after the `app.get` method, type the code as shown in Code Snippet 4.

Code Snippet 4:

```
//Sending the result based on the marks using app.post()
var_app.post('/result', (req, res) => {
 let stuName = req.body.stuName;
 let stuMarks = req.body.stuMarks;
 if (stuMarks > 60) {
 res.send('<p> Student Name: ' + stuName + '<p> Student
Marks: ' + stuMarks + '<p> Student Result: Passed');
 }
 else {
 res.send('<p> Student Name: ' + stuName + '<p> Student
Marks: ' + stuMarks + '<p> Student Result: Failed');
 }
 console.log(response);
})
```

5. In the same folder where Express.js is installed, save the file as `server.js`.
6. To execute the application, in the Command Prompt window, navigate to the folder where Express.js is installed.
7. At the command prompt, run the command as:

```
node server.js
```

The command executes and creates the Web server, as shown in Figure 6.6.

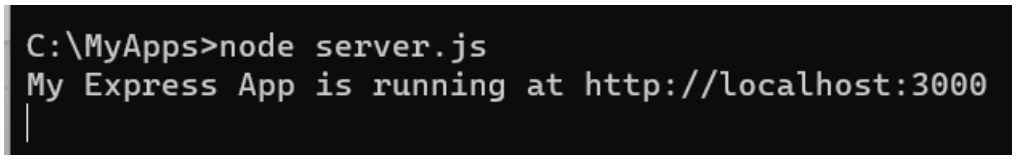


Figure 6.6: Web Server Created

8. Open a browser.
9. Navigate to the URL, `http://localhost:3000`.

   The `StudentResult.html` page opens as shown in Figure 6.7.

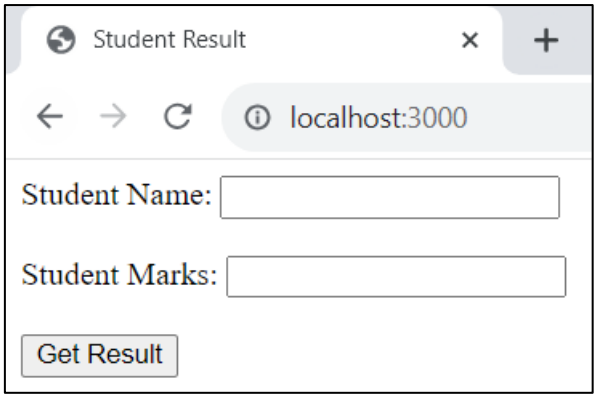


Figure 6.7: `StudentResult` Page

10. Enter the student's name and marks as shown in Figure 6.8.

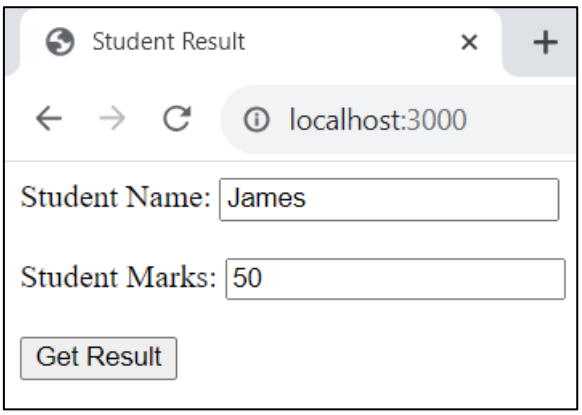


Figure 6.8: Student Details Entered

11. Click **Get Result**.

    The result is displayed as shown in Figure 6.9.

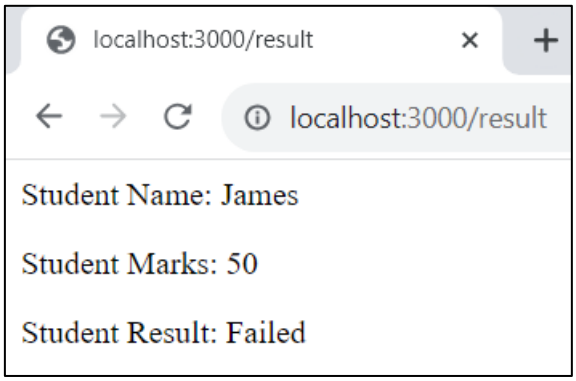


Figure 6.9: Result: Failed

12. Open another browser tab and navigate to the URL: `http:/localhost:3000/`.

13. Enter the student details as shown in Figure 6.10.

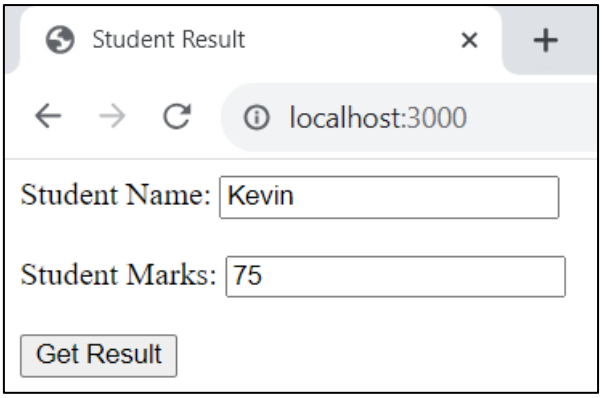


Figure 6.10: Student Details Entered

14. Click **Get Result**.

The result is displayed as shown in Figure 6.11.

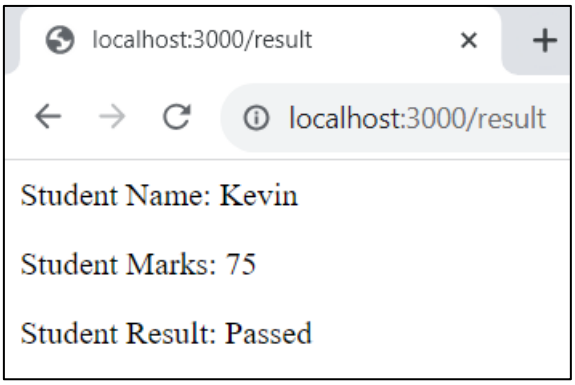


Figure 6.11: Result: Passed

The characteristic of the middleware is to move to the next stacked request after responding to the current request. To understand this better, perform these steps:

1. Open a file using VS Code or Notepad and add the code given in Code Snippet 5.

Code Snippet 5:

```
const var_express = require('express');
const var_app = var_express();
const var_port = 3000;
// Middleware function
let var_count = 1;
var_app.use((req, res, next) => {
 console.log('This is a simple middleware that logs a message
for request:' + var_count);
var_count = var_count+1;
 next(); // Call next to pass control to the next middleware
or route handler
});
```

```
var_app.get('/', (req, res) => res.send('Welcome to My
Express App'));
var_app.listen(var_port, () => {
 console.log(`My Express App is running at
http://localhost:${var_port}`);
});
```

2. In the folder where Express.js is installed, save the file as `middlewareSample.js`.
3. Open the Command Prompt window and navigate to the folder where Express.js is installed.
4. At the command prompt, run the command as:

```
node middlewareSample.js
```

The command starts a server as shown in Figure 6.12.

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
```

Figure 6.12: Server Started

5. Open a browser window and navigate to the URL, `http://localhost:3000`.

The Web page shows the message as shown in Figure 6.13.

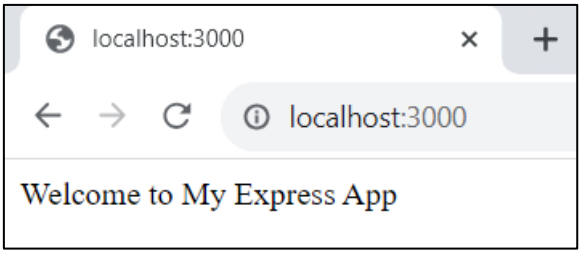


Figure 6.13: Browser Output at Port 3000

6. Switch to the Command Prompt window.

The message is displayed as shown in Figure 6.14.

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
```

Figure 6.14: Message Logged for First Request

7. Switch to the browser window and refresh the page.

8. Switch back to the Command Prompt window.

   The second message is displayed as shown in Figure 6.15.

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
This is a simple middleware that logs a message for request:2
```

Figure 6.15: Message Logged for Second Request

With every refresh, the request number is incremented as shown in Figure 6.16.

```
C:\MyApps>node middlewareSample.js
My Express App is running at http://localhost:3000
This is a simple middleware that logs a message for request:1
This is a simple middleware that logs a message for request:2
This is a simple middleware that logs a message for request:3
This is a simple middleware that logs a message for request:4
This is a simple middleware that logs a message for request:5
```

Figure 6.16: Message Logged for Subsequent Requests

This shows how the middleware moves to the next request in the queue.

6.8 Summary

- Express.js is a framework that is used for server-side programming with Node.js runtime environment.
- Express.js can handle different types of HTTP requests, generate dynamic HTML, provide middleware functions, and serve static files.
- The Express.js framework is made up of three interconnected components—view, controller, and model—spread across three layers—application, service, and data access, respectively.
- The view component is responsible for generating the HTML content that is sent to the client.
- The model component handles the data and the business logic and interacts with the underlying data sources.
- The controller component provides the logic that acts as a bridge between the view and model components.
- The order in which components of Express.js are involved is server instance → router → middleware → handler functions → templating/static file serving.
- Express.js is a minimalist, extensible framework that is easy to use and adopt.
- It provides enhanced performance and supports several databases and templating engines.
- The `request` object handles the HTTP requests coming from the clients and provides information, such as the URL, HTTP method, HTTP header, request body, and client IP.
- The `response` object is responsible for generating the responses for the requests and sending them back to the client.
- Express.js provides the `app.get` function to define handlers for `GET` requests and the `app.post` function to define handlers for `POST` requests.

Test Your Knowledge

1. What is the purpose of middleware?

   a) It is a bridge between databases
   b) It is a single library
   c) It is a tool that can only be used in React
   d) It is used to handle HTTP requests and responses

2. You are working on a Web application. Your client wants you to add Express.js to your application. What is the correct sequence of components you will use in your application?

   i. Middleware
   ii. Router
   iii. Server instance
   iv. Handler functions
   v. Templating/Static file serving

   a) i, ii, iii, iv, v
   b) iv, v, iii, ii, i
   c) iii, ii, i, iv, v
   d) iii, i, ii, iv, v

3. Consider the given code. Add the missing code to complete the application.

```
const var_express = require('express');
const var_app = express();
const port = 3000;

// Add missing code here
app.listen(port, () => {
 console.log(`My Express App is running at
http://localhost:${port}`);
});
```

   a) `app.get(req, res) => res.send('This is an example of express.'));`
   b) `app.get('/html', (req, res) => res.send('This is an example of express.'));`
   c) `app.get('/', (req, res) => res.send('This is an example of express.'));`
   d) `app.get('/', (res) => res.send('This is an example of express.'));`

4. What is the purpose of `url` property in HTTP?

    a) It stores encoding data
    b) It stores route parameters
    c) It stores HTTP headers
    d) It stores the complete URL included in the request sent by the client

5. What will be the output of the given code:

```
const var_express = require('express');
const var_app = express();
const port = 3000;

app.get('/', (req, res) => res.send('Alan loves to play
cricket'));

app.listen(3000, () => {
 console.log(`Express App is running on port 3000.`);
});
```

    a) It will display `Alan loves to play cricket`
    b) It will display `Express App is running on port 3000`
    c) It will display an error
    d) None of these

Answers to Test Your Knowledge

1	d
2	c
3	c
4	d
5	a

Try It Yourself

1. Create a simple Web application for a company to display their Employee Information such as Employee Name, Employee ID, and Employee Designation using `Express.js` as backend.

    a) Install `Express.js` and the `body-parser Package`.
    b) Create a `Web` page as `EmployeeInfo.html`.
    c) Create an `app.js` file.
    d) Inside the `app.js` file create an Express Web server.
    e) Create a body-parser instance.
    f) Configure the Web server to listen on port `3000`.
    g) Serve the `EmployeeInfo.html` page using the `app.get` method.
    h) Send the result based on the employee information using the `app.post` method.
    i) Display the result on the Web page.

2. Create a simple Web server using Express and use a middleware in the code. Display the message, `My first Web server working successfully`, for each request.